LECTURE 6

# Neural Networks: Hidden Layers and Backpropagation

Reference Text

Part I: why hidden layers — the single neuron, the XOR wall, non-linear activations, learned feature maps and universal approximation. Part II: backpropagation by hand — one forward and one backward pass through a 2–2–1 sigmoid network

AK DHAMIJA, Researcher & Instructor, AI & Machine Learning

dhamija.ak@gmail.com • 9811453163

Unit 4: Simulation Strategies in Warfare — AI & Machine Learning in Warfare

This reference text expands every slide of the lecture into full explanations, derivations, worked examples and figures. Read it alongside the slides; work the "check your understanding" boxes before looking at the answers.

# Contents

**1 How to Use This Text** **3**

**I Why Hidden Layers?** **3**

**2 The Artificial Neuron** **3**
2.1 Definition . . . 3
2.2 One neuron draws one straight line . . . 3
2.3 The perceptron learning rule . . . 4

**3 The XOR Wall** **5**
3.1 The problem . . . 5
3.2 Proof that no single neuron computes XOR . . . 5
3.3 What logistic regression actually does on XOR . . . 5
3.4 History . . . 6

**4 Width and Linear Depth Are Not Enough** **6**
4.1 One wide layer is still linear . . . 6
4.2 Stacked linear layers collapse into one . . . 7

**5 Non-Linear Activation Functions** **7**

**6 A Hidden Layer Solves XOR** **8**
6.1 The network . . . 8
6.2 Reading the hidden units . . . 8

**7 Hidden Layers as Learned Feature Maps** **8**
7.1 Re-mapping the data . . . 9
7.2 Advantages of learned feature maps . . . 9

**8 How Far Can Hidden Layers Go?** **10**
8.1 Each ReLU adds a bend . . . 10
8.2 The universal approximation theorem . . . 10
8.3 Why depth: features of features . . . 11

**9 Counting Parameters** **11**

**10 Check Your Understanding (Part I)** **11**

**II Backpropagation by Hand** **12**

**11 The Network and Its Notation** **13**

11.1 Architecture and given values . . . 13
11.2 Notation: superscript for the layer, subscript for the index . . . 13

**12 The Two Building Blocks 14**
12.1 The sigmoid activation . . . 14
12.2 The binary cross-entropy loss . . . 14

**13 The Forward Pass 15**
13.1 F1 — Hidden pre-activations . . . 15
13.2 F2 — Hidden activations . . . 15
13.3 F3 — Output pre-activation . . . 16
13.4 F4 — Prediction . . . 16
13.5 F5 — Loss . . . 16

**14 The Backward Pass 17**
14.1 The chain rule and why "backpropagation" . . . 17
14.2 B1 — The output error signal . . . 17
14.3 B2 — Output-layer gradients . . . 18
14.4 B3 and B4 — Error signals at the hidden layer . . . 18
14.5 B5 — Hidden-layer gradients . . . 19
14.6 The backward flow at a glance . . . 19

**15 Verifying the Gradients 20**
15.1 Numerical gradient check . . . 20
15.2 One gradient-descent step . . . 20
15.3 Why gradients vanish in deep sigmoid networks . . . 21

**16 Defence and Wargaming Perspective 21**

**17 Common Mistakes 21**

**18 Summary of Equations 22**

**19 Check Your Understanding (Part II) 22**

# 1 How to Use This Text

This reference text accompanies the Lecture 6 slides. It has two parts that answer the two questions every analyst should be able to answer about a neural network.

- **Part I — Why hidden layers?** What can a single artificial neuron do, what can it never do, and why does a *hidden* layer with a *non-linear* activation remove that limit? The answer turns on one small problem, XOR, and one big idea: hidden layers are *learned feature maps*.
- **Part II — Backpropagation by hand.** Given that hidden layers are needed, how are their weights learned? One example is pushed forward and backward through a 2–2–1 network, with every number computed.

Coloured boxes mark the different kinds of material: **Definition** boxes give terms you must be able to state; **Derivation** boxes show where formulas come from; **Worked example** boxes carry complete calculations; **Defence & wargaming** boxes connect ideas to practice; **Pitfall** boxes describe common errors; **Check your understanding** boxes pose questions whose answers follow.

Every network in the rest of the course is built from the ideas of this lecture: the convolutional network of Lecture 7, the transformers of Lectures 8 and 9, and the deep Q-networks and policy networks of Lectures 10–12. Lecture 1 introduced neurons and backpropagation at headline level; this lecture does the mathematics.

# Part I
# Why Hidden Layers?

## 2 The Artificial Neuron

### 2.1 Definition

**Definition — artificial neuron**

An artificial neuron with inputs $x = (x_1, \ldots, x_d)$ computes a **pre-activation** $z = w \cdot x + b = \sum_{i=1}^{d} w_i x_i + b$ and an **output** $a = f(z)$. The numbers $w_i$ are **weights**, $b$ is the **bias**, and $f$ is the **activation function**.

The weights say how much each input matters and in which direction; the bias shifts the point at which the neuron "switches on". Logistic regression (Lecture 2) is exactly one neuron with a sigmoid activation, $f(z) = 1/(1 + e^{-z})$, read as a probability.

### 2.2 One neuron draws one straight line

To classify, we threshold the output: predict class 1 when $f(z)$ exceeds some level $\tau$. For every common activation — the step, the sigmoid, tanh, ReLU on its active side — $f$ is *monotonic increasing*, so $f(z) > \tau$ is equivalent to $z > c$ for some constant $c$ (for the sigmoid with $\tau = 0.5$, $c = 0$). Absorbing $c$ into the bias, the decision rule is always

$$\hat{y} = 1 \iff w_1 x_1 + w_2 x_2 + b > 0.$$

**Derivation — the decision boundary is a straight line**

The boundary is the set where $w_1x_1 + w_2x_2 + b = 0$. If $w_2 \neq 0$ this rearranges to

$$x_2 = -\frac{w_1}{w_2}\,x_1 - \frac{b}{w_2},$$

a straight line with slope $-w_1/w_2$ and intercept $-b/w_2$. The weight vector $w = (w_1, w_2)$ is perpendicular to the line and points towards the class-1 side. In $d$ dimensions the boundary $w \cdot x + b = 0$ is a flat *hyperplane*.

Training a single neuron can move and rotate this line, but never bend it. Problems whose classes can be split by one straight line (one hyperplane) are called **linearly separable**.

**Worked example — AND and OR with one neuron**

Use a step output, $\hat{y} = 1$ if $z > 0$.

| | | AND: $w = (1,1)$, $b = -1.5$ | | OR: $w = (1,1)$, $b = -0.5$ | |
|---|---|---|---|---|---|
| $x_1$ | $x_2$ | $z$ | $\hat{y}$ | $z$ | $\hat{y}$ |
| 0 | 0 | −1.5 | 0 | −0.5 | 0 |
| 0 | 1 | −0.5 | 0 | 0.5 | 1 |
| 1 | 0 | −0.5 | 0 | 0.5 | 1 |
| 1 | 1 | 0.5 | 1 | 1.5 | 1 |

The AND boundary is the line $x_1 + x_2 = 1.5$; the OR boundary is $x_1 + x_2 = 0.5$. Both are correct on all four inputs. Changing only the bias moves the line parallel to itself and turns AND into OR.

## 2.3 The perceptron learning rule

Rosenblatt's perceptron (1958) learned such lines from examples. For each training example with label $y \in \{0, 1\}$ and prediction $\hat{y}$, it updates

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(y - \hat{y}).$$

Nothing changes when the prediction is right; a missed positive pulls the line towards the example, a false alarm pushes it away. The **perceptron convergence theorem** guarantees that this finds a separating line in finitely many steps *if one exists*.

**Worked example — learning AND with the perceptron rule**

Start from $w = (0, 0)$, $b = 0$, $\eta = 1$, step output with $\hat{y} = 1$ iff $z > 0$. Visit the examples in the order $(0,0), (0,1), (1,0), (1,1)$.
Epoch 1: $(0,0)$: $z = 0 \Rightarrow \hat{y} = 0$, correct. $(0,1)$: $z = 0$, correct. $(1,0)$: correct. $(1,1)$: $z = 0 \Rightarrow \hat{y} = 0$, but $y = 1$, so $w = (1,1)$, $b = 1$.
Epoch 2: $(0,0)$: $z = 1 \Rightarrow \hat{y} = 1$, wrong: $b = 0$. $(0,1)$: $z = 1$, wrong: $w = (1,0)$, $b = -1$. $(1,0)$: $z = 0$, correct. $(1,1)$: $z = 0$, wrong: $w = (2,1)$, $b = 0$.
Epoch 3: $(0,0)$: $z = 0$, correct. $(0,1)$: $z = 1$, wrong: $w = (2,0)$, $b = -1$. $(1,0)$: $z = 1$, wrong: $w = (1,0)$, $b = -2$. $(1,1)$: $z = -1$, wrong: $w = (2,1)$, $b = -1$.
Epoch 4: $(0,0)$: $z = -1$, correct. $(0,1)$: $z = 0$, correct. $(1,0)$: $z = 1$, wrong: $w = (1,1)$, $b = -2$. $(1,1)$: $z = 0$, wrong: $w = (2,2)$, $b = -1$.
Epoch 5: $(0,0)$: $z = -1$, correct. $(0,1)$: $z = 1$, wrong: $w = (2,1)$, $b = -2$. $(1,0)$: $z = 0$, correct. $(1,1)$: $z = 1$, correct.
Epoch 6: $z = -2, -1, 0, 1$ for the four inputs: all correct, and training stops with $w = (2,1)$,

$b = -2$, the line $2x_1 + x_2 = 2$. It differs from the hand-chosen $x_1 + x_2 = 1.5$ but separates the data equally well: a separable problem has many solutions.

# 3 The XOR Wall

## 3.1 The problem

**Exclusive OR** (XOR) outputs 1 when exactly one input is 1. Plotted on the unit square, class 1 sits at $(0, 1)$ and $(1, 0)$ and class 0 at $(0, 0)$ and $(1, 1)$: the classes occupy opposite corners. Try any straight line: whichever side contains both class-1 corners also contains at least one class-0 corner.

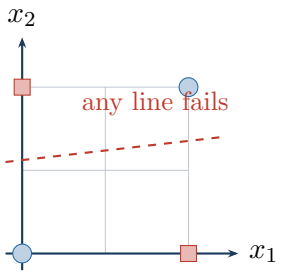



Figure 1: XOR. Squares are class 1, circles class 0.

## 3.2 Proof that no single neuron computes XOR

**Derivation — XOR is not linearly separable**

Suppose weights $w_1, w_2$ and bias $b$ existed such that the output is 1 exactly when $z = w_1 x_1 + w_2 x_2 + b > 0$. The four rows of the truth table require

$$\begin{aligned} (0,0) \mapsto 0: &\quad b \le 0 && (1)\\ (1,0) \mapsto 1: &\quad w_1 + b > 0 && (2)\\ (0,1) \mapsto 1: &\quad w_2 + b > 0 && (3)\\ (1,1) \mapsto 0: &\quad w_1 + w_2 + b \le 0 && (4) \end{aligned}$$

Adding (2) and (3) gives $w_1 + w_2 + 2b > 0$, so $w_1 + w_2 + b > -b$. By (1), $-b \ge 0$, so $w_1 + w_2 + b > 0$, contradicting (4). No such weights exist. ■

The proof does not depend on which monotonic activation is used, so the same impossibility holds for logistic regression, a sigmoid neuron or a ReLU neuron.

## 3.3 What logistic regression actually does on XOR

It is instructive to see what a single sigmoid neuron *learns* when trained on XOR with the cross-entropy loss.

**Worked example — the best single neuron is useless**

Take $w = (0, 0)$, $b = 0$: every input gets $p = \sigma(0) = 0.5$. The cross-entropy gradient of a sigmoid neuron is $\sum_n (p_n - y_n) x_n$ for the weights and $\sum_n (p_n - y_n)$ for the bias (derived in

Part II). Here

$$\sum_n (p_n - y_n)x_n = (0.5)(0,0) + (-0.5)(0,1) + (-0.5)(1,0) + (0.5)(1,1) = (0,0),$$
$$\sum_n (p_n - y_n) = 0.5 - 0.5 - 0.5 + 0.5 = 0.$$

The gradient is zero. Because the logistic-regression loss is convex, this stationary point is the *global* minimum: the best single neuron predicts 0.5 for every input, with loss $\ln 2 \approx 0.693$ per example. It has learned nothing.

### 3.4 History

Minsky and Papert's book *Perceptrons* (1969) made this limitation famous. Multi-layer networks could in principle solve XOR, but nobody had a practical way to train their hidden weights, and research funding moved elsewhere. The revival came when Rumelhart, Hinton and Williams (1986) popularised **backpropagation** — the subject of Part II.

**Defence & wargaming application — an XOR-shaped threat picture**

Consider air tracks described by speed and altitude (an illustrative, simplified picture). Fast-and-low tracks match sea-skimming or cruise-missile profiles (hostile); slow-and-high tracks match loitering surveillance platforms (hostile); slow-and-low tracks are often own helicopters, and fast-and-high tracks airliners or own jets (routine). Neither speed alone nor altitude alone separates hostile from routine; only their *combination* does — the XOR pattern. A single-neuron classifier on these two inputs can do no better than the majority class. Real data contain many such interactions: "either A or B but not both", a target inside a ring of clutter, interleaved patterns.

## 4 Width and Linear Depth Are Not Enough

### 4.1 One wide layer is still linear

A layer of $K$ neurons computes $\hat{y}_k = f(w_k \cdot x + b_k)$ for $k = 1, \ldots, K$. Each output draws its own straight line on the *same raw inputs*, and no output sees another's answer. For XOR every one of the $K$ neurons faces the same impossibility proof. With a softmax over the outputs (multinomial logistic regression), class $k$ is predicted where $w_k \cdot x + b_k$ is largest; the regions are separated by straight lines and each region is *convex* — so no class can occupy two opposite corners. Width adds lines but cannot *reason with* them ("inside line 1 AND outside line 2"). That requires a second stage whose inputs are the first stage's outputs.

### 4.2 Stacked linear layers collapse into one

**Derivation — composition of linear maps**

Two layers with the identity activation compute

$$\hat{y} = W_2(W_1 x + b_1) + b_2 = \underbrace{(W_2 W_1)}_{W} x + \underbrace{(W_2 b_1 + b_2)}_{b}.$$

The product of two matrices is a matrix and the rest is a vector, so the two-layer network is exactly a one-layer network with weights $W$ and bias $b$. By induction, any number of linear layers collapses into one.

**Worked example — checking the collapse numerically**

$W_1 = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$, $b_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $W_2 = (1 \quad -1)$, $b_2 = 0.5$. Collapsed: $W = W_2 W_1 = (1 \cdot 1 + (-1) \cdot 0,\ 1 \cdot 2 + (-1) \cdot 1) = (1,\ 1)$ and $b = W_2 b_1 + b_2 = 1 + 0 + 0.5 = 1.5$.
At $x = (1, 1)$: the two-layer network gives $h = (1+2+1,\ 1) = (4, 1)$ and $\hat{y} = 4 - 1 + 0.5 = 3.5$; the collapsed layer gives $1 + 1 + 1.5 = 3.5$.
At $x = (2, -1)$: $h = (2-2+1,\ -1) = (1, -1)$, $\hat{y} = 1+1+0.5 = 2.5$; collapsed: $2-1+1.5 = 2.5$.

**Key point**

Depth without non-linearity is an illusion. The hidden layer must *bend* its output, or the whole network remains one straight boundary. This is why every convolution in a CNN (Lecture 7) and every feed-forward block in a transformer (Lecture 9) is followed by a non-linear activation.

## 5 Non-Linear Activation Functions

| Name | $f(z)$ | $f'(z)$ | Notes |
|---|---|---|---|
| Step | 1 if $z > 0$, else 0 | 0 (undefined at 0) | perceptron; no usable gradient |
| Sigmoid | $1/(1+e^{-z})$ | $\sigma(z)(1-\sigma(z)) \le 0.25$ | range $(0, 1)$; probability output |
| Tanh | $\tanh z$ | $1 - \tanh^2 z \le 1$ | range $(-1, 1)$; zero-centred |
| ReLU | $\max(0, z)$ | 1 if $z > 0$, 0 if $z < 0$ | default for hidden layers |

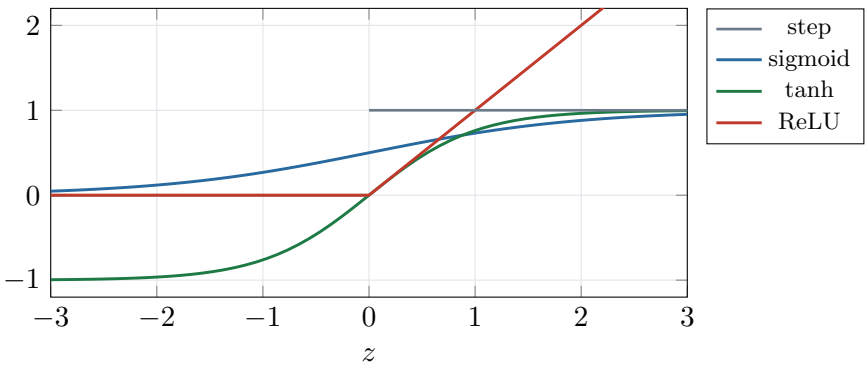



Figure 2: four activation functions.

The only requirement for breaking the collapse is that $f$ is *not* of the form $az + b$. ReLU — two straight pieces joined at a kink — is enough. It is preferred in hidden layers because it is cheap

and its derivative is exactly 1 for active units, so gradients do not shrink as they pass back through many layers; the sigmoid's derivative is at most 0.25, which causes the *vanishing-gradient* problem examined in Part II. The step function has zero derivative almost everywhere, so gradient-based training cannot use it.

# 6 A Hidden Layer Solves XOR

## 6.1 The network

Take two hidden ReLU units and a linear output:

$$h = \mathrm{ReLU}(Wx + c), \qquad \hat{y} = w^{\mathsf{T}} h, \qquad W = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \; c = \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \; w = \begin{pmatrix} 1 \\ -2 \end{pmatrix}.$$

So $h_1 = \mathrm{ReLU}(x_1 + x_2)$, $h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$ and $\hat{y} = h_1 - 2h_2$. This is the classic solution in Goodfellow, Bengio and Courville, *Deep Learning* (2016), §6.1.

**Worked example — the forward pass for all four inputs**

| $x_1$ | $x_2$ | $z_1$ | $z_2$ | $h_1$ | $h_2$ | $\hat{y} = h_1 - 2h_2$ | XOR |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | −1 | 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 2 | 1 | 2 | 1 | 0 | 0 |

All four outputs are correct. In row 1, ReLU clips $z_2 = -1$ to 0; without the clip, $h_2 = -1$ and $\hat{y} = 0 - 2(-1) = 2$, which is wrong.

## 6.2 Reading the hidden units

$h_1$ counts *how many* inputs are on (an "OR-like" detector); $h_2$ is non-zero only when *both* are on (an "AND" detector). The output says "at least one, but subtract heavily if both": XOR = OR AND NOT AND. The hidden layer computes the pieces; the output combines them — the combining that a single layer could not do.

**Worked example — the same network without ReLU**

Replace ReLU by the identity: $\hat{y} = z_1 - 2z_2 = (x_1 + x_2) - 2(x_1 + x_2 - 1) = -x_1 - x_2 + 2$. The outputs for $(0,0), (0,1), (1,0), (1,1)$ are $2, 1, 1, 0$. The network has collapsed to one straight line (one linear function), exactly as Section 4.2 predicts, and it no longer computes XOR.

# 7 Hidden Layers as Learned Feature Maps

## 7.1 Re-mapping the data

The hidden layer is a function $x \mapsto h(x)$ that sends each input point to a new point. For the XOR network:

$$(0,0) \mapsto (0,0), \qquad (0,1) \mapsto (1,0), \qquad (1,0) \mapsto (1,0), \qquad (1,1) \mapsto (2,1).$$

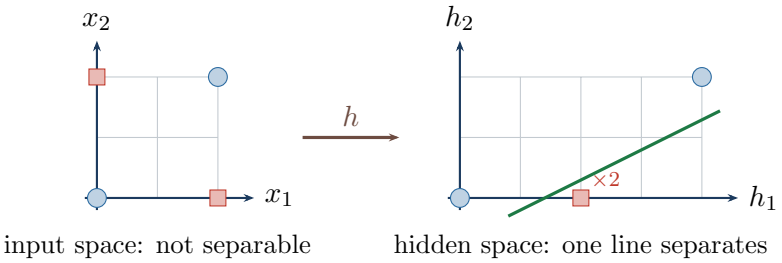



Figure 3: the hidden layer moves both class-1 points to the same spot.

In the hidden coordinates both class-1 inputs land on the same point $(1,0)$, and a single straight line — the output neuron, $h_1 - 2h_2 = 0.5$ — separates the classes. The output neuron is still only a line, but a line *in the new space*.

**Definition — feature map and representation learning**

The activations of a hidden layer, viewed as a function of the input, form a **feature map**: a learned change of coordinates. **Representation learning** means choosing this map by training, so that the final problem becomes easy (often linearly separable), instead of having an engineer hand-craft the features.

In a dense layer, a hidden unit's feature is one number per example. In a CNN (Lecture 7) each filter's feature map is a whole image — the filter's response at every location. In a transformer (Lectures 8–9) the features are context-aware vectors for each token.

## 7.2 Advantages of learned feature maps

- **Automatic feature engineering:** the network discovers useful intermediate quantities ("how many on?", "both on?") from labels alone.
- **Re-usable:** the same features serve many outputs and tasks; this is the basis of transfer learning.
- **Compact:** a few hundred features can summarise thousands of raw inputs.
- **Inspectable:** feature maps can be visualised to see what the network responds to (with care — see below).
- **Hierarchical:** deeper layers combine shallower features into parts and objects.

**Defence & wargaming application — hidden units as learned indicators**

On the air-track data of Section 3, a trained hidden layer could develop units that fire for regions of the speed–altitude plane: "fast and low", "slow and high", "slow and low", "fast and high". The output then needs only a weighted vote — positive weights on the two hostile indicators, negative on the two routine ones — which is linear in the indicator space. Add radar cross-section, heading-change rate and IFF response as inputs, and the hidden layer can learn combinations no analyst has written down. **Reality check:** real hidden units rarely align with neat human labels; features are usually spread across many units. Interpreting them needs care, and high-consequence calls need a human analyst.

# 8 How Far Can Hidden Layers Go?

## 8.1 Each ReLU adds a bend

A ReLU network computes a continuous *piecewise-linear* function: each hidden unit contributes one kink. Combining three units gives a bump:

**Worked example — a bump from three ReLUs**

$f(x) = \mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$.
For $x \le 0$ all terms vanish: $f = 0$. For $0 \le x \le 1$: $f = x$, so $f(0.5) = 0.5$, $f(1) = 1$. For $1 \le x \le 2$: $f = x - 2(x-1) = 2 - x$, so $f(1.5) = 0.5$, $f(2) = 0$. For $x \ge 2$: $f = x - 2(x-1) + (x-2) = 0$. The result is a triangle of height 1 on $[0, 2]$.

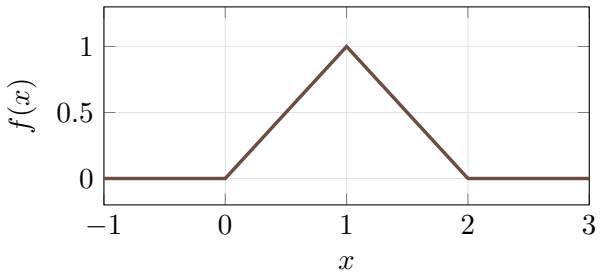



Figure 4: three hidden ReLU units make a bump.

Scaled and shifted bumps can be added to trace any continuous curve, like drawing a curve from many short straight segments. In two dimensions each hidden unit contributes one line, and the output combines them ("inside line 1 AND line 2 AND ..."); six units can enclose a target inside a ring of clutter, and more units give a smoother region.

## 8.2 The universal approximation theorem

**Definition — universal approximation (Cybenko 1989; Hornik 1991)**

A feed-forward network with one hidden layer of sufficiently many units with a suitable non-linear activation (sigmoid, ReLU and many others) can approximate any continuous function on a bounded region to any desired accuracy.

So the answer to "*can* a network represent my problem?" is almost always yes — if it has a hidden layer with a non-linear activation. The fine print matters:

1. **How many units?** The theorem says enough units *exist*; for complicated functions one hidden layer may need astronomically many.
2. **Can training find them?** It says nothing about whether gradient descent will find the right weights.
3. **Will it generalise?** Fitting the training data is not the same as predicting new data well (Lecture 1's overfitting and distribution shift).

### 8.3 Why depth: features of features

A deep network computes a composition $f_3(f_2(f_1(x)))$. Each layer reuses everything below it, so a feature learned once (an edge, a "low-altitude" detector) serves many higher features. The number of distinct linear regions a ReLU network can carve out grows *exponentially* with depth but only polynomially with width (Montúfar et al., 2014): depth buys complexity cheaply. An analogy is a staff process — sensor operators report contacts, analysts group them into formations, and the intelligence officer infers intent; each level builds on the one below.

Depth has a cost: deeper networks are harder to train, because gradients must pass back through many layers (Part II, Section 15.3). ReLU activations, careful initialisation, normalisation layers and residual connections are the tools that made 100-layer networks practical.

**Pitfall — five misconceptions about hidden layers**

**"More neurons in one layer solve everything."** Without a hidden layer, each output is still one straight line. **"More layers always help."** Only if separated by non-linear activations, and only if the network can still be trained. **"The activation is a detail."** It is the entire source of the network's power beyond linear regression. **"Hidden features are designed by engineers."** They are learned by backpropagation from labels. **"Universal approximation means any network will work."** It guarantees existence, not size, trainability or generalisation.

## 9 Counting Parameters

A dense layer from $n$ inputs to $m$ units has $nm$ weights and $m$ biases: $(n+1)m$ parameters. The XOR network has $(2+1)\times 2 = 6$ hidden parameters and 2 output weights (no output bias): 8 in all. The 2–2–1 network of Part II has $(2+1)\times 2+(2+1)\times 1 = 9$ parameters — the nine gradients computed there.

**Worked example — a threat classifier**

A classifier with 20 track features, hidden layers of 64 and 32 ReLU units and a 5-class softmax output has $(20+1)\times 64+(64+1)\times 32+(32+1)\times 5 = 1{,}344+2{,}080+165 = 3{,}589$ parameters. Lecture 7 shows how convolution keeps image networks small despite having thousands of inputs.

## 10 Check Your Understanding (Part I)

**Check your understanding — 1**

Give weights and a bias for a single step neuron that computes NAND (output 0 only when both inputs are 1).

**Answer.** NAND is NOT AND, so negate the AND neuron: $w = (-1,-1)$, $b = 1.5$. Then $z = 1.5,\ 0.5,\ 0.5,\ -0.5$ for $(0,0),(0,1),(1,0),(1,1)$, giving outputs $1,1,1,0$.

**Check your understanding — 2**

Which inequality in the XOR proof uses the $(0,0)$ row, and why is it needed?

**Answer.** Inequality (1), $b \le 0$. Adding (2) and (3) only gives $w_1 + w_2 + b > -b$; we need $-b \ge 0$, from (1), to conclude $w_1 + w_2 + b > 0$ and contradict (4).

**Check your understanding — 3**

Collapse $W_1 = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix}$, $b_1 = 0$, $W_2 = (1 \;\; 1)$, $b_2 = -1$ into one layer.

**Answer.** $W = W_2 W_1 = (2+1,\; 0+1) = (3,\; 1)$ and $b = W_2 b_1 + b_2 = -1$, so $\hat{y} = 3x_1 + x_2 - 1$.

**Check your understanding — 4**

In the XOR network, what does $\hat{y}$ become for $(0, 0)$ if ReLU is replaced by the identity?

**Answer.** $z_1 = 0$, $z_2 = -1$, so $\hat{y} = z_1 - 2z_2 = 0 + 2 = 2$ instead of 0. The linear network computes $-x_1 - x_2 + 2$ and fails.

**Check your understanding — 5**

Name two advantages of learned feature maps over hand-engineered features.

**Answer.** Any two of: they are discovered automatically from labels; they are reusable across tasks (transfer learning); they compress many raw inputs into a few informative quantities; they can be visualised; they build hierarchically into more abstract features.

**Check your understanding — 6**

A single sigmoid neuron trained on XOR outputs 0.5 for every input. Is this a training failure that more epochs would fix?

**Answer.** No. Section 3.3 shows the gradient is exactly zero at $w = 0, b = 0$, and the loss is convex, so this is the global optimum for a single neuron. Only a hidden layer can do better.

## Part II

# Backpropagation by Hand

# 11 The Network and Its Notation

## 11.1 Architecture and given values

The network has two inputs, one hidden layer of two sigmoid units, and one sigmoid output unit. It is trained with the binary cross-entropy (BCE) loss on a single example whose correct label is $y = 1$.

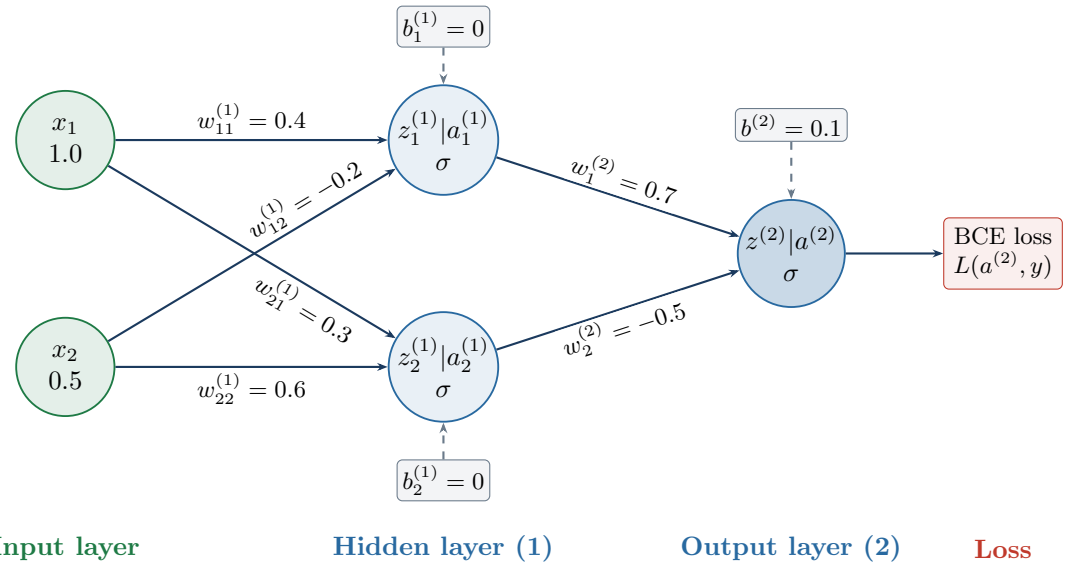



Figure 5. The 2–2–1 network with every parameter shown on its connection. Dashed arrows are biases.

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix}, \quad y = 1, \qquad W^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{pmatrix} = \begin{pmatrix} 0.4 & -0.2 \\ 0.3 & 0.6 \end{pmatrix}, \quad b^{(1)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

$$w^{(2)} = \begin{pmatrix} w_1^{(2)} & w_2^{(2)} \end{pmatrix} = \begin{pmatrix} 0.7 & -0.5 \end{pmatrix}, \qquad b^{(2)} = 0.1.$$

## 11.2 Notation: superscript for the layer, subscript for the index

**Definition — layered notation**

The **layer number** is always written as a parenthesised superscript and the **matrix or vector indices** as subscripts.

- $w_{jk}^{(1)}$ is the weight *into* hidden unit $j$ *from* input $x_k$: row $j$, column $k$ of $W^{(1)}$.
- $b_j^{(1)}$, $z_j^{(1)}$ and $a_j^{(1)}$ are the bias, pre-activation and activation of hidden unit $j$.
- $w_j^{(2)}$ is the weight from hidden unit $j$ to the output unit; $b^{(2)}$, $z^{(2)}$ and $a^{(2)} = \hat{y}$ belong to the output unit.
- $\delta_j^{(l)} = \partial L / \partial z_j^{(l)}$ is the **error signal** at unit $j$ of layer $l$.

The parentheses matter: $w^{(2)}$ means "the weights of layer 2", whereas $w^2$ would mean "$w$ squared". The order of the two subscripts also matters: the first names the receiving unit, the second the sending unit. With this convention one row of $W^{(1)}$ holds every weight entering one hidden unit, so the product $W^{(1)}x$ produces the vector of hidden pre-activations directly.

# 12 The Two Building Blocks

## 12.1 The sigmoid activation

**Definition — logistic sigmoid**

$\sigma(z) = \dfrac{1}{1+e^{-z}}$ maps any real number $z$ into the open interval $(0,1)$.

**Derivation — the derivative of the sigmoid**

Write $\sigma(z) = (1+e^{-z})^{-1}$ and differentiate with the chain rule:

$$\sigma'(z) = -(1+e^{-z})^{-2}\cdot(-e^{-z}) = \frac{e^{-z}}{(1+e^{-z})^2} = \underbrace{\frac{1}{1+e^{-z}}}_{\sigma(z)}\cdot\underbrace{\frac{e^{-z}}{1+e^{-z}}}_{1-\sigma(z)}.$$

Hence $\sigma'(z) = \sigma(z)\big(1-\sigma(z)\big)$. The derivative can be computed from the activation alone — no exponential needs to be re-evaluated in the backward pass.

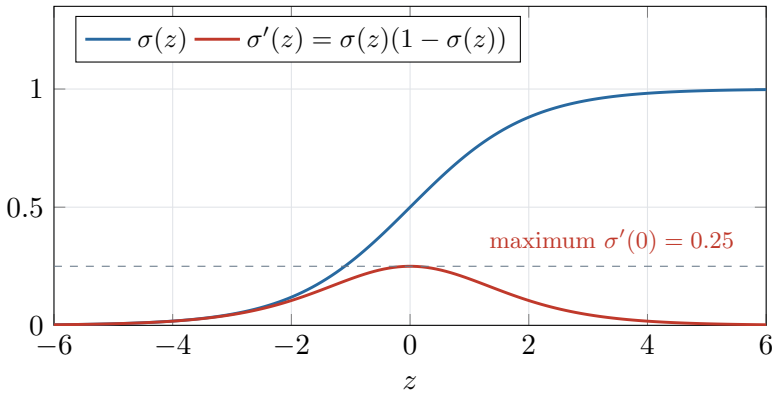



Figure 6. The sigmoid and its derivative. The slope never exceeds 0.25.

**Key point**

Because $\sigma'(z) \le 0.25$, every sigmoid layer that the error signal passes through multiplies it by at most one quarter. This is the origin of the *vanishing-gradient problem* in deep sigmoid networks, and the main reason modern hidden layers use ReLU-type activations.

## 12.2 The binary cross-entropy loss

**Definition — binary cross-entropy**

For a label $y \in \{0,1\}$ and a predicted probability $a^{(2)} = \hat{y} \in (0,1)$,

$$L = -\big[\,y \ln a^{(2)} + (1-y)\ln\big(1-a^{(2)}\big)\big].$$

The BCE is the negative log-likelihood of a Bernoulli label: if the network says "the probability of class 1 is $\hat{y}$", the probability it assigns to the label actually observed is $\hat{y}^{\,y}(1-\hat{y})^{1-y}$, and $L$ is minus the logarithm of that. Minimising $L$ therefore maximises the likelihood of the training labels. When $y=1$ only the first term survives, $L = -\ln\hat{y}$; when $y=0$ only the second, $L = -\ln(1-\hat{y})$.

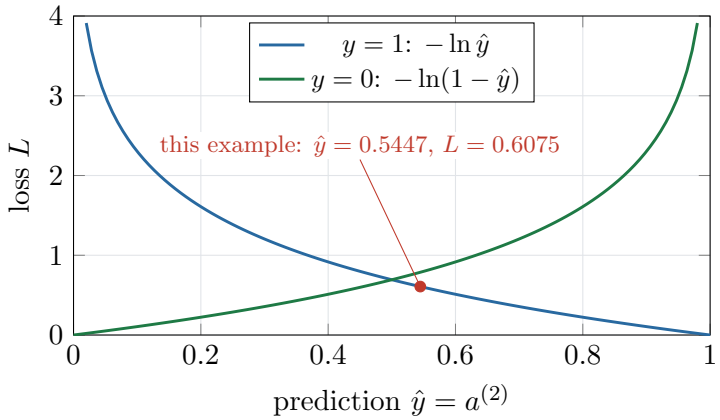



Figure 7. The BCE loss rises without limit as a prediction becomes confidently wrong.

**Defence & wargaming application — the cost of confident errors**

Suppose a missile-warning classifier is shown a genuine threat ($y = 1$). An undecided output $\hat{y} = 0.5$ costs $L = -\ln 0.5 = 0.693$. A confident but wrong output $\hat{y} = 0.01$ costs $L = -\ln 0.01 = 4.605$, more than six times as much. BCE therefore trains the network hardest on exactly the errors that are most dangerous operationally: the threats it is sure are harmless.

# 13 The Forward Pass

The forward pass moves left to right through Figure 5, computing each layer from the one before it. Every intermediate value is *kept*, because the backward pass will need it.

## 13.1 F1 — Hidden pre-activations

Each hidden pre-activation is a weighted sum of the inputs plus a bias: $z^{(1)} = W^{(1)}x + b^{(1)}$, or unit by unit $z_j^{(1)} = \sum_k w_{jk}^{(1)} x_k + b_j^{(1)}$.

**Worked example — F1**

$$z_1^{(1)} = w_{11}^{(1)}x_1 + w_{12}^{(1)}x_2 + b_1^{(1)} = (0.4)(1.0) + (-0.2)(0.5) + 0 = 0.4 - 0.1 = \mathbf{0.300000}$$
$$z_2^{(1)} = w_{21}^{(1)}x_1 + w_{22}^{(1)}x_2 + b_2^{(1)} = (0.3)(1.0) + (0.6)(0.5) + 0 = 0.3 + 0.3 = \mathbf{0.600000}$$

In matrix form: $\begin{pmatrix} 0.4 & -0.2 \\ 0.3 & 0.6 \end{pmatrix} \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.3 \\ 0.6 \end{pmatrix}$.

## 13.2 F2 — Hidden activations

**Worked example — F2**

$$a_1^{(1)} = \sigma(0.3) = \frac{1}{1 + e^{-0.3}} = \frac{1}{1 + 0.740818} = \mathbf{0.574443}$$
$$a_2^{(1)} = \sigma(0.6) = \frac{1}{1 + e^{-0.6}} = \frac{1}{1 + 0.548812} = \mathbf{0.645656}$$

Quick check: near $z = 0$, $\sigma(z) \approx 0.5 + z/4$, giving 0.575 and 0.650 — close to the exact values, as expected for small $z$.

## 13.3 F3 — Output pre-activation

**Worked example — F3**

$$z^{(2)} = w_1^{(2)} a_1^{(1)} + w_2^{(2)} a_2^{(1)} + b^{(2)} = (0.7)(0.574443) + (-0.5)(0.645656) + 0.1$$
$$= 0.402110 - 0.322828 + 0.1 = \mathbf{0.179282}$$

Notice the second term is negative: hidden unit 2 is active ($a_2^{(1)} = 0.65$) but its outgoing weight is negative, so it *pulls the output down*. This single observation explains a sign in the backward pass (Section 14.4).

## 13.4 F4 — Prediction

**Worked example — F4**

$$a^{(2)} = \hat{y} = \sigma(0.179282) = \frac{1}{1 + e^{-0.179282}} = \frac{1}{1 + 0.835871} = \mathbf{0.544701}$$

## 13.5 F5 — Loss

**Worked example — F5**

With $y = 1$ the $(1 - y)$ term is zero:

$$L = -\big[\, 1 \cdot \ln(0.544701) + 0 \cdot \ln(0.455299) \,\big] = -\ln(0.544701) = \mathbf{0.607519}$$

**Table 1. Forward-pass results**

| | | | |
|---|---|---|---|
| $z_1^{(1)}$ | 0.300000 | $a_1^{(1)}$ | 0.574443 |
| $z_2^{(1)}$ | 0.600000 | $a_2^{(1)}$ | 0.645656 |
| $z^{(2)}$ | 0.179282 | $a^{(2)} = \hat{y}$ | 0.544701 |
| Loss $L = 0.607519$ | | | |

The network gives a 54.5% probability to an example whose true label is 1: barely better than a coin-flip. The rest of this text finds out how each parameter should change to raise that probability.

# 14 The Backward Pass

## 14.1 The chain rule and why "backpropagation"

The loss depends on a weight in the first layer only through a chain of intermediate quantities. For $w_{11}^{(1)}$ the chain is

$$w_{11}^{(1)} \xrightarrow{\times x_1} z_1^{(1)} \xrightarrow{\sigma} a_1^{(1)} \xrightarrow{\times w_1^{(2)}} z^{(2)} \xrightarrow{\sigma} a^{(2)} \xrightarrow{\text{BCE}} L,$$

and the chain rule multiplies the local derivative of every link:

$$\frac{\partial L}{\partial w_{11}^{(1)}} = \underbrace{\frac{\partial L}{\partial a^{(2)}}\frac{\partial a^{(2)}}{\partial z^{(2)}}}_{\delta^{(2)}} \frac{\partial z^{(2)}}{\partial a_1^{(1)}} \frac{\partial a_1^{(1)}}{\partial z_1^{(1)}} \frac{\partial z_1^{(1)}}{\partial w_{11}^{(1)}} = \underbrace{\delta^{(2)}\, w_1^{(2)}\, \sigma'(z_1^{(1)})}_{\delta_1^{(1)}}\, x_1.$$

**Key point**

Backpropagation is the chain rule *plus reuse*. The first two factors, $\delta^{(2)}$, are common to every parameter in the network, so they are computed once. The first four factors, $\delta_1^{(1)}$, are common to every weight entering hidden unit 1, so they too are computed once. Working from the loss backwards guarantees that each shared factor is available when it is needed. This is why the cost of computing *all* gradients is only about twice the cost of one forward pass.

## 14.2 B1 — The output error signal

**Derivation — $\delta^{(2)} = a^{(2)} - y$ for a sigmoid output with BCE loss**

Differentiate the loss with respect to the prediction:

$$\frac{\partial L}{\partial a^{(2)}} = -\frac{y}{a^{(2)}} + \frac{1-y}{1-a^{(2)}} = \frac{-y(1-a^{(2)}) + (1-y)a^{(2)}}{a^{(2)}(1-a^{(2)})} = \frac{a^{(2)} - y}{a^{(2)}\big(1-a^{(2)}\big)}.$$

The sigmoid contributes $\partial a^{(2)}/\partial z^{(2)} = a^{(2)}(1-a^{(2)})$. Multiplying,

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = \frac{a^{(2)} - y}{a^{(2)}(1-a^{(2)})} \cdot a^{(2)}(1-a^{(2)}) = a^{(2)} - y.$$

**Worked example — B1**

$$\delta^{(2)} = a^{(2)} - y = 0.544701 - 1 = \mathbf{-0.455299}$$

**Pitfall — using squared error with a sigmoid output**

Had we used $L = \frac{1}{2}(\hat{y} - y)^2$, the sigmoid factor would *not* cancel: $\delta^{(2)} = (\hat{y} - y)\,\hat{y}(1-\hat{y}) = (-0.455299)(0.544701)(0.455299) = -0.112915$, four times weaker. Worse, if the network were confidently wrong ($\hat{y} \approx 0$ for $y = 1$), the factor $\hat{y}(1-\hat{y}) \approx 0$ would almost silence the gradient precisely when learning is most needed. Sigmoid outputs should be paired with BCE.

## 14.3 B2 — Output-layer gradients

Because $z^{(2)} = w_1^{(2)} a_1^{(1)} + w_2^{(2)} a_2^{(1)} + b^{(2)}$, we have $\partial z^{(2)}/\partial w_j^{(2)} = a_j^{(1)}$ and $\partial z^{(2)}/\partial b^{(2)} = 1$. Hence $\partial L/\partial w_j^{(2)} = \delta^{(2)} a_j^{(1)}$ and $\partial L/\partial b^{(2)} = \delta^{(2)}$.

**Worked example — B2**

$$\frac{\partial L}{\partial w_1^{(2)}} = \delta^{(2)} a_1^{(1)} = (-0.455299)(0.574443) = \mathbf{-0.261543}$$
$$\frac{\partial L}{\partial w_2^{(2)}} = \delta^{(2)} a_2^{(1)} = (-0.455299)(0.645656) = \mathbf{-0.293967}$$
$$\frac{\partial L}{\partial b^{(2)}} = \delta^{(2)} = \mathbf{-0.455299}$$

All three are negative. Gradient *descent* subtracts the gradient, so it will *increase* $w_1^{(2)}$, $w_2^{(2)}$ and $b^{(2)}$; each of these increases raises $z^{(2)}$ and therefore pushes $\hat{y}$ towards the target 1. The rule to remember is **gradient = (error signal at the receiving unit) × (activation on the sending side)**; a bias always sends the value 1.

## 14.4 B3 and B4 — Error signals at the hidden layer

The error reaches hidden unit $j$ through the single weight $w_j^{(2)}$ and is then scaled by the slope of that unit's sigmoid:

$$\delta_j^{(1)} = \frac{\partial L}{\partial z_j^{(1)}} = \delta^{(2)}\, w_j^{(2)}\, \sigma'(z_j^{(1)}), \qquad \sigma'(z_j^{(1)}) = a_j^{(1)}\big(1 - a_j^{(1)}\big).$$

**Worked example — B3 — sigmoid slopes**

$$\sigma'(z_1^{(1)}) = (0.574443)(1 - 0.574443) = (0.574443)(0.425557) = \mathbf{0.244458}$$
$$\sigma'(z_2^{(1)}) = (0.645656)(1 - 0.645656) = (0.645656)(0.354344) = \mathbf{0.228784}$$

**Worked example — B4 — hidden error signals**

$$\delta_1^{(1)} = (-0.455299)(0.7)(0.244458) = (-0.318709)(0.244458) = \mathbf{-0.077911}$$
$$\delta_2^{(1)} = (-0.455299)(-0.5)(0.228784) = (0.227650)(0.228784) = \mathbf{0.052083}$$

The positive sign of $\delta_2^{(1)}$ is the backward-pass echo of the observation made in F3. Hidden unit 2 feeds the output through a *negative* weight, so increasing its activity lowers $\hat{y}$ — the wrong direction for a positive example. The gradient therefore tells unit 2 to become less active. Note also the size of the signals: $|\delta^{(2)}| = 0.455$ but $|\delta_j^{(1)}| \le 0.078$. One weight smaller than 1 and one sigmoid slope at most 0.25 have reduced the signal about six-fold in a single layer.

## 14.5 B5 — Hidden-layer gradients

Since $z_j^{(1)} = \sum_k w_{jk}^{(1)} x_k + b_j^{(1)}$, we have $\partial z_j^{(1)}/\partial w_{jk}^{(1)} = x_k$ and $\partial z_j^{(1)}/\partial b_j^{(1)} = 1$:

$$\frac{\partial L}{\partial w_{jk}^{(1)}} = \delta_j^{(1)}\, x_k, \qquad \frac{\partial L}{\partial b_j^{(1)}} = \delta_j^{(1)}.$$

**Worked example — B5**

$$\frac{\partial L}{\partial w_{11}^{(1)}} = (-0.077911)(1.0) = \mathbf{-0.077911}, \qquad \frac{\partial L}{\partial w_{12}^{(1)}} = (-0.077911)(0.5) = \mathbf{-0.038956},$$
$$\frac{\partial L}{\partial w_{21}^{(1)}} = (0.052083)(1.0) = \mathbf{0.052083}, \qquad \frac{\partial L}{\partial w_{22}^{(1)}} = (0.052083)(0.5) = \mathbf{0.026041},$$
$$\frac{\partial L}{\partial b_1^{(1)}} = \mathbf{-0.077911}, \qquad \frac{\partial L}{\partial b_2^{(1)}} = \mathbf{0.052083}.$$

In matrix form the whole weight gradient is one outer product:

$$\nabla_{W^{(1)}} L = \delta^{(1)} x^{\mathsf{T}} = \begin{pmatrix} -0.077911 \\ 0.052083 \end{pmatrix} \begin{pmatrix} 1.0 & 0.5 \end{pmatrix} = \begin{pmatrix} -0.077911 & -0.038956 \\ 0.052083 & 0.026041 \end{pmatrix}.$$

## 14.6 The backward flow at a glance

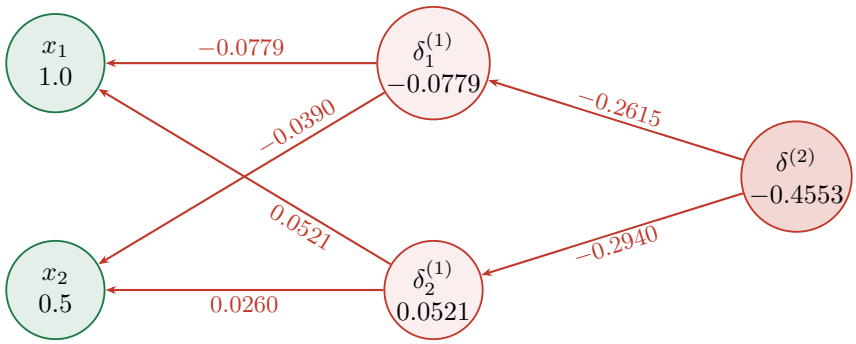



Figure 8. Backward flow. Nodes carry $\delta = \partial L/\partial z$; edges carry $\partial L/\partial w$ for that connection.

**Table 2. All gradients**

| Parameter | $\partial L/\partial(\cdot)$ | Parameter | $\partial L/\partial(\cdot)$ |
|---|---|---|---|
| $w_{11}^{(1)}$ | −0.077911 | $w_1^{(2)}$ | −0.261543 |
| $w_{12}^{(1)}$ | −0.038956 | $w_2^{(2)}$ | −0.293967 |
| $w_{21}^{(1)}$ | 0.052083 | $b^{(2)}$ | −0.455299 |
| $w_{22}^{(1)}$ | 0.026041 | $\delta^{(2)}$ | −0.455299 |
| $b_1^{(1)}$ | −0.077911 | $\delta_1^{(1)}$ | −0.077911 |
| $b_2^{(1)}$ | 0.052083 | $\delta_2^{(1)}$ | 0.052083 |

# 15 Verifying the Gradients

## 15.1 Numerical gradient check

A hand-derived or hand-coded gradient should always be checked against a finite-difference estimate,

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta+\varepsilon) - L(\theta-\varepsilon)}{2\varepsilon}.$$

**Worked example — central-difference check of $\partial L/\partial w_{11}^{(1)}$ with $\varepsilon = 0.01$**

Re-running the forward pass with $w_{11}^{(1)} = 0.41$ gives $L = 0.6067406$; with $w_{11}^{(1)} = 0.39$ it gives $L = 0.6082988$. Then

$$\frac{0.6067406 - 0.6082988}{0.02} = \frac{-0.0015582}{0.02} = -0.077911,$$

in agreement with the backpropagated value to six decimal places. The same check for all nine parameters reproduces every entry of Table 2.

Deep-learning libraries compute gradients automatically, but gradient checking remains the standard test when a new layer or loss is implemented by hand.

## 15.2 One gradient-descent step

The problem does not specify a learning rate; for illustration take $\eta = 0.5$ and apply $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$ to every parameter *simultaneously*, using gradients computed from the old values.

**Worked example — update with $\eta = 0.5$**

$$W^{(1)} \leftarrow \begin{pmatrix} 0.4 - 0.5(-0.077911) & -0.2 - 0.5(-0.038956) \\ 0.3 - 0.5(0.052083) & 0.6 - 0.5(0.026041) \end{pmatrix} = \begin{pmatrix} 0.438956 & -0.180522 \\ 0.273959 & 0.586979 \end{pmatrix}$$

$$b^{(1)} \leftarrow (0.038956,\ -0.026041), \qquad w^{(2)} \leftarrow (0.830772,\ -0.353017), \qquad b^{(2)} \leftarrow 0.327650$$

Repeating the forward pass with these values gives $\hat{y} = 0.645519$ and $L = 0.437701$. The prediction has moved towards the target and the loss has fallen from 0.6075 to 0.4377, confirming that the gradients point in the right direction.

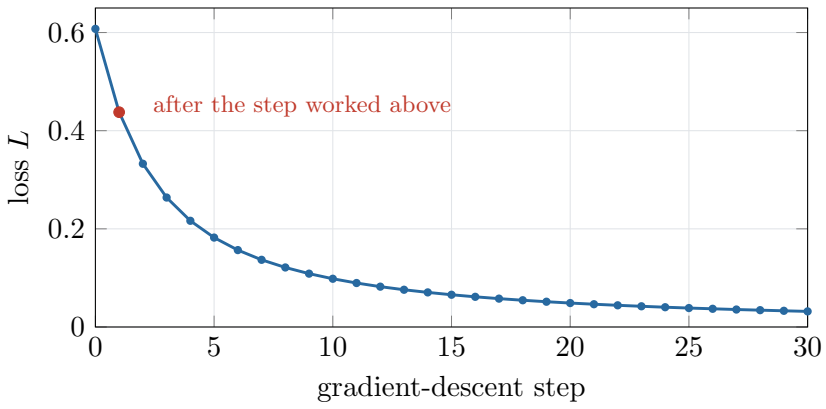



Figure 9. Repeating the same step on the same example ($\eta = 0.5$): after 10 steps $\hat{y} = 0.906$, after 30 steps $\hat{y} = 0.969$.

### 15.3 Why gradients vanish in deep sigmoid networks

The hidden error signal $\delta_j^{(1)} = \delta^{(2)} w_j^{(2)} \sigma'(z_j^{(1)})$ shows the general pattern: each layer the error passes back through multiplies it by a weight and by the derivative of that layer's activation. With $L$ sigmoid layers the signal reaching the first layer contains a product of $L$ factors $\sigma'(z) \le 0.25$. Even with weights of size 1, ten layers shrink it by at least $0.25^{10} \approx 9.5 \times 10^{-7}$: the early layers barely learn. This is the **vanishing-gradient problem**, and it is why the ReLU of Part I (derivative exactly 1 for active units) replaced the sigmoid in hidden layers, and why very deep networks add *residual connections* $y = x + F(x)$, whose derivative contains a direct term 1 that carries the gradient past each block. Lecture 9 meets residual connections again in every transformer block.

## 16 Defence and Wargaming Perspective

**Defence & wargaming application — a neural threat classifier**

Replace the two inputs by two radar-track features — say, normalised closing speed and a normalised IFF-response score — and read the output as $P(\text{hostile} \mid x)$. The weights then have an operational meaning. A negative weight such as $w_{12}^{(1)} = -0.2$ means the second feature *inhibits* the first hidden detector, as a strong IFF reply should inhibit a "threat" detector. The gradient signs found above tell us how one hostile training track reshapes the classifier: the detector that supports a hostile call (unit 1) is strengthened, while the detector that argues against it (unit 2) is weakened.

**Defence & wargaming application — why vanishing gradients matter operationally**

Deep sigmoid networks trained on sensor data can stall: the early layers, which learn the most basic features of the signal, receive gradients shrunk by a factor of 4 or more per layer and barely change. A model that appears to train (its output layer adapts) may in fact be relying on poorly-learned features that do not transfer to new theatres — a direct link to the distribution-shift failures discussed in Lecture 1.

## 17 Common Mistakes

**Pitfall — forgetting $\sigma'$ at a hidden unit**

$\delta_j^{(1)} = \delta^{(2)} w_j^{(2)}$ is wrong. The error must also pass through the hidden sigmoid: multiply by $a_j^{(1)}(1 - a_j^{(1)})$.

**Pitfall — adding $\sigma'$ at the output when using BCE**

With a sigmoid output and BCE loss the factor $a^{(2)}(1 - a^{(2)})$ has already cancelled. Writing $\delta^{(2)} = (\hat{y} - y)\sigma'(z^{(2)})$ counts it twice.

**Pitfall — swapping the indices**

$w_{jk}^{(1)}$ goes *to* unit $j$ *from* input $k$, and its gradient is $\delta_j^{(1)} x_k$. Writing $\delta_k^{(1)} x_j$ transposes the gradient matrix.

**Pitfall — updating weights in the middle of the backward pass**

$\delta_j^{(1)}$ uses the *old* value of $w_j^{(2)}$. Compute every gradient first, then update every parameter together.

**Pitfall — the sign of the step**

Gradient descent is $\theta - \eta \nabla L$. Adding the gradient climbs the loss surface. After any hand calculation, re-run the forward pass: for a small enough $\eta$ the loss must fall.

## 18 Summary of Equations

| | |
|---|---|
| **Forward** | $z^{(1)} = W^{(1)}x + b^{(1)}, \quad a^{(1)} = \sigma(z^{(1)}), \quad z^{(2)} = w^{(2)}a^{(1)} + b^{(2)}, \quad a^{(2)} = \sigma(z^{(2)})$ |
| **Loss** | $L = -\big[y \ln a^{(2)} + (1-y)\ln(1-a^{(2)})\big]$ |
| **Output error** | $\delta^{(2)} = a^{(2)} - y$ |
| **Hidden error** | $\delta_j^{(1)} = \delta^{(2)}\, w_j^{(2)}\, a_j^{(1)}(1-a_j^{(1)})$ |
| **Gradients** | $\partial L/\partial w_j^{(2)} = \delta^{(2)} a_j^{(1)}, \quad \partial L/\partial b^{(2)} = \delta^{(2)}, \quad \partial L/\partial w_{jk}^{(1)} = \delta_j^{(1)} x_k, \quad \partial L/\partial b_j^{(1)} = \delta_j^{(1)}$ |
| **Update** | $\theta \leftarrow \theta - \eta\, \partial L/\partial \theta$ |

## 19 Check Your Understanding (Part II)

**Check your understanding — 1**

If the label were $y = 0$ instead of 1, with everything else unchanged, what would $\delta^{(2)}$ be, and in which direction would $b^{(2)}$ move?

**Answer.** $\delta^{(2)} = a^{(2)} - y = 0.544701 - 0 = 0.544701$. The gradient $\partial L/\partial b^{(2)} = 0.544701$ is positive, so gradient descent *decreases* $b^{(2)}$, lowering $\hat{y}$ towards 0.

**Check your understanding — 2**

If $w_2^{(2)}$ were $+0.5$ instead of $-0.5$, what would happen to the weights entering hidden unit 2?

**Answer.** $\delta_2^{(1)} = (-0.455299)(+0.5)(0.228784) = -0.052083$, now negative, so $\partial L/\partial w_{21}^{(1)}$ and $\partial L/\partial w_{22}^{(1)}$ become negative and gradient descent *increases* both weights. Unit 2 would now support the positive output, so the network makes it more active — the same treatment unit 1 receives.

**Check your understanding — 3**

Why is $|\partial L/\partial w_{12}^{(1)}|$ exactly half of $|\partial L/\partial w_{11}^{(1)}|$?

**Answer.** Both weights enter the same hidden unit and so share the factor $\delta_1^{(1)}$; their gradients are $\delta_1^{(1)} x_2$ and $\delta_1^{(1)} x_1$, and $x_2 = 0.5$ is half of $x_1 = 1.0$. Weights entering one unit always have gradients in proportion to their inputs.

**Check your understanding — 4**

What is the largest possible value of $\sigma'(z)$, and why does it matter for deep networks?

**Answer.** $\sigma'(z) = \sigma(z)(1-\sigma(z))$ is largest when $\sigma(z) = 0.5$, i.e. $z = 0$, where it equals 0.25. Each sigmoid layer therefore scales the backward signal by at most 0.25 (times the weight); across ten layers the factor is at most $0.25^{10} \approx 10^{-6}$, so early layers learn extremely slowly — the vanishing-gradient problem.